



DEI

DEPARTAMENTO
DE ENGENHARIA INFORMÁTICA

TÉCNICO LISBOA

Methodology for Distributed Computing

Parallel and Distributed Computing

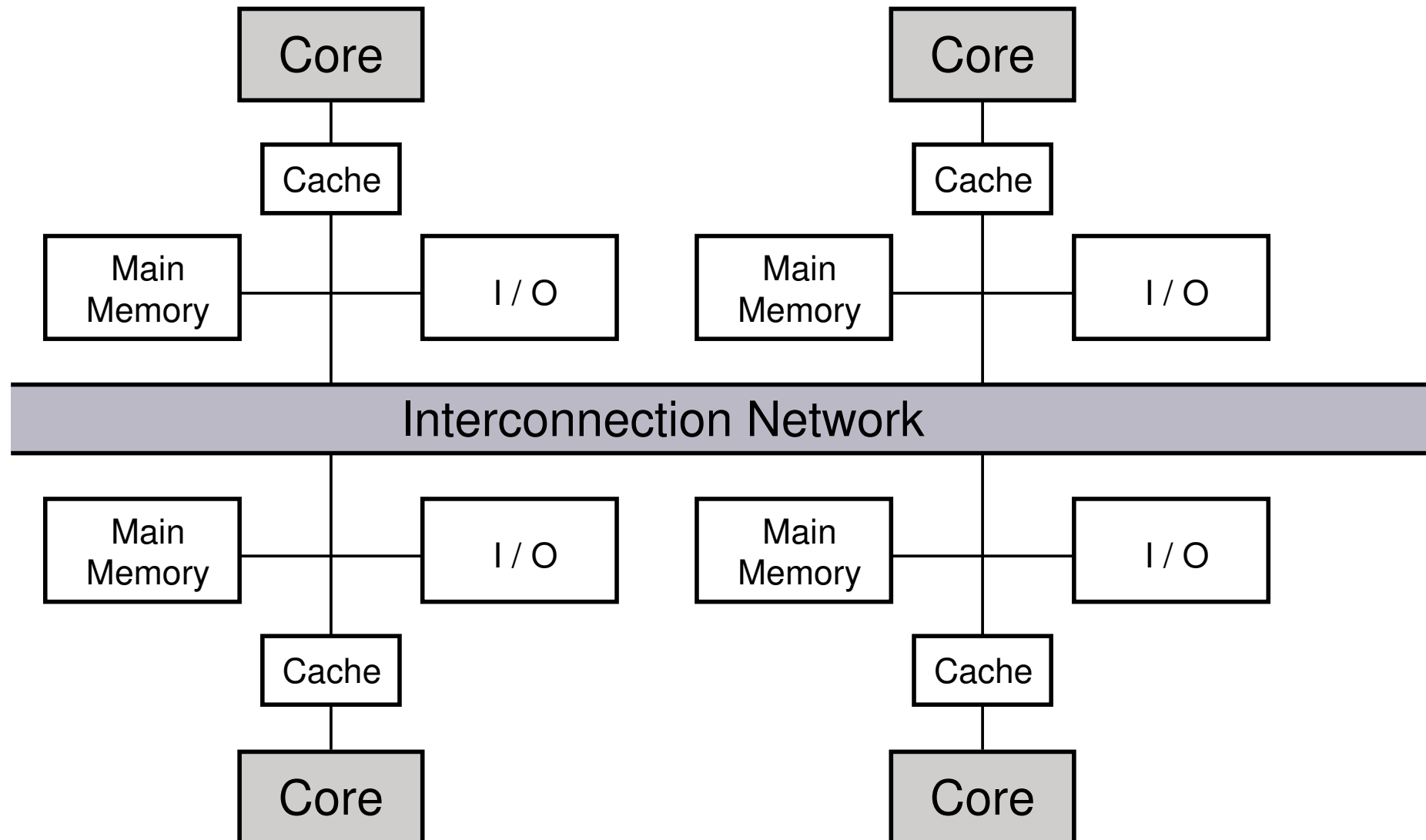
José Monteiro

Monday, 9 March 2026

Outline

- Programming Distributed-memory Systems
- Foster's Design Methodology
 - Partitioning
 - Communication
 - Agglomeration
 - Mapping
- Application Examples
 - Boundary value problem
 - Finding the maximum
 - n-body problem

Distributed-Memory Systems



Overheads in Parallel Programs

- Task creation / finish
- Data transfer
- Task coordination (communication, synchronization)
- Load balancing

Shared vs. Distributed Memory

- Overheads very different between architectures

	Task Start / Finish	Data Transfer	Load Balancing	Synchronization
Shared Memory	▼	▼	=	▲
Distributed Memory	▲	▲	=	▼

Shared vs. Distributed Memory

	Shared Memory	Message-Passing
Tasks	More dynamic creation of tasks, hence these can be more fine-grained	Typically all tasks active until end , hence requires more coarse-grained tasks
Data	Data partition not an issue when defining tasks; however caution when accessing shared data: avoid races using mutual-exclusive regions	Data partition is critical for the performance of the application

In both cases:

- Minimize synchronization points
- Be careful about load balancing

Architecture vs Programming Model

Programming Model

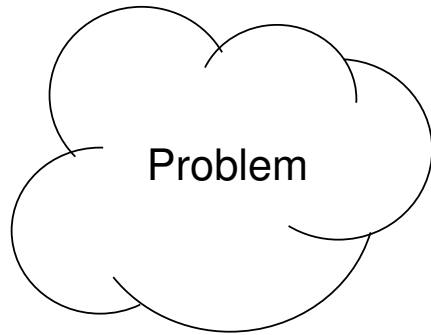
	Single Address Space	Multiple Address Spaces
Shared-memory		
Distributed-memory		

Hardware

Distributed Shared Memory

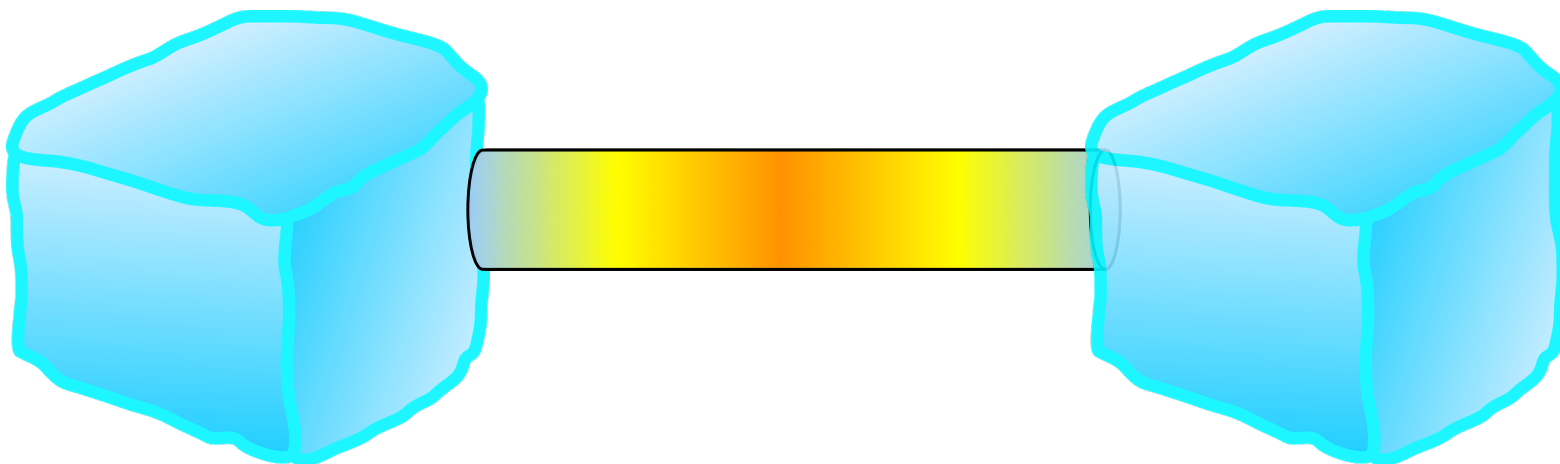
- “Shared-memory” across machines requires special support
 - Program memory defined within a process
 - A process “lives” within an operating system
 - Shared address spaces must be mapped between processes in different machines
 - Data migration is then automatic
 - Easier to program, very poor performance...
 - **Message-passing** is the programming model of choice for distributed-memory systems
- ➔ Data location/migration a concern from the start!

Foster's Design Methodology



Boundary Value Problem

- Problem: Determine the evolution of the temperature of a rod
 - Rod of length 1
 - Uniform material
 - Both ends of the rod at 0°C
 - Insulated except at the ends
 - Initial temperature of a point at distance x is $100^{\circ}\sin(\pi x)$



Boundary Value Problem

- A partial differential equation (PDE) governs the temperature of the rod in time
 - Typically too complicated to solve analytically
- Finite difference methods can be used to obtain an approximate solution to these complex problems
- Discretize space and time
 - Unit-length rod divided into n sections of length h
 - Time from 0 to P divided into m periods of length k

Boundary Value Problem

- We obtain a grid $n \times m$ of temperatures $T(x,t)$

$$T(x,t) = r \cdot T(x-1,t-1) + (1 - 2r) \cdot T(x,t-1) + r \cdot T(x+1,t-1)$$

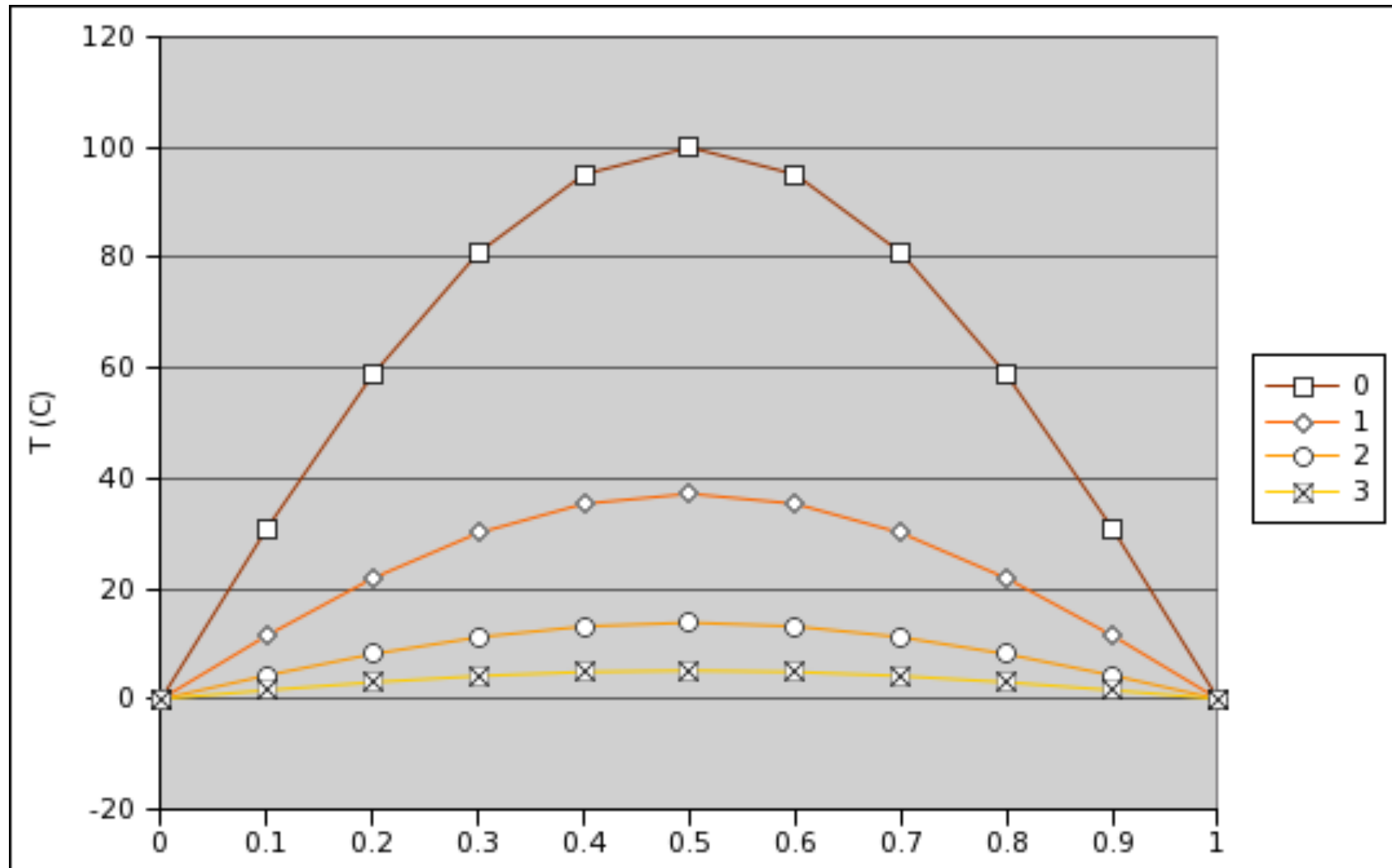
where $r = k / h^2$

:

$$\begin{array}{ccc} x=0 & x=2h & \dots \\ x=h & x=3h & \end{array}$$

$$x=1=nh$$

Boundary Value Problem

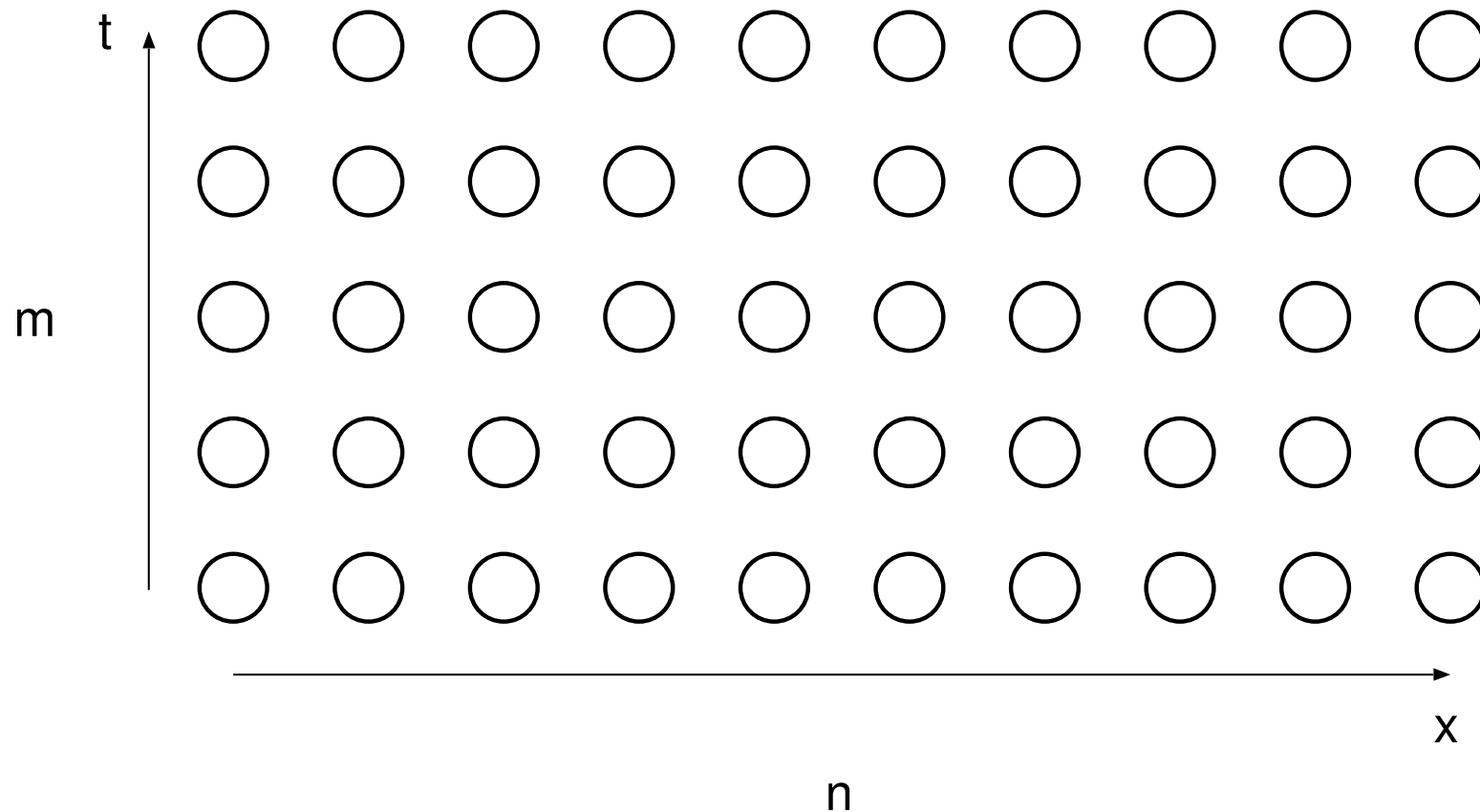


Boundary Value Problem

- Partitioning

- Make each $T(x,t)$ computation a primitive task

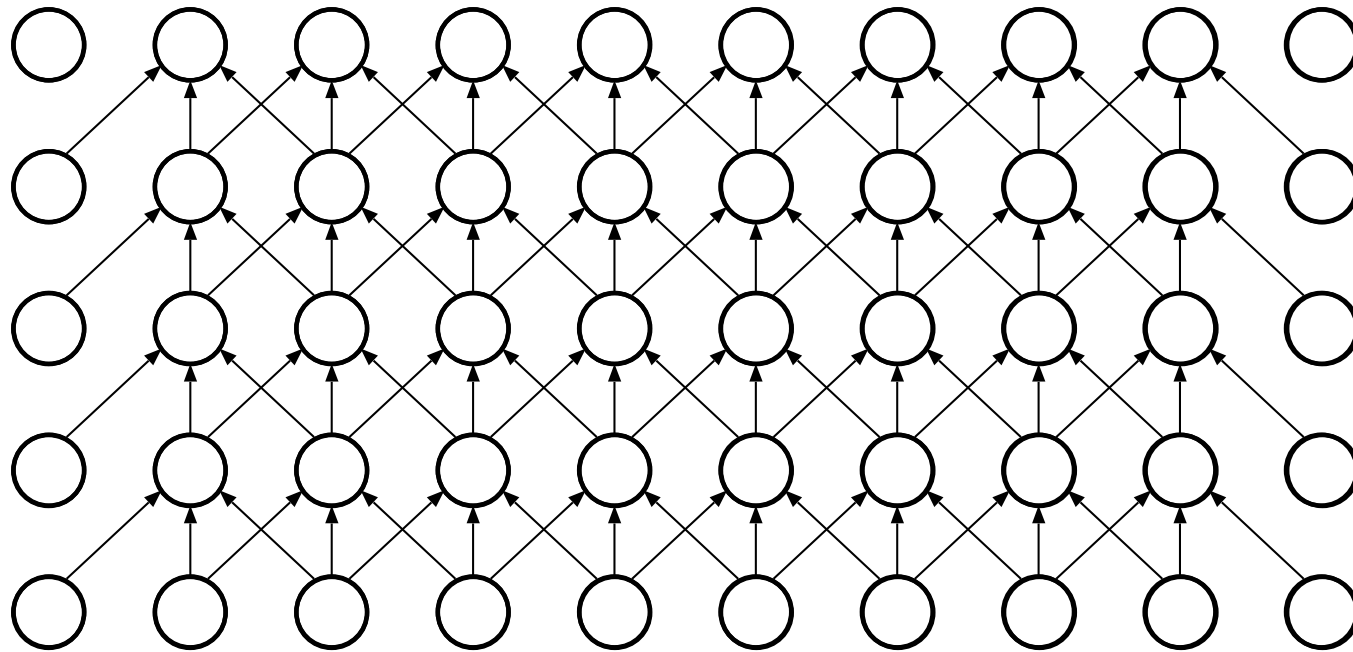
- ➔ 2-dimensional domain decomposition



Boundary Value Problem

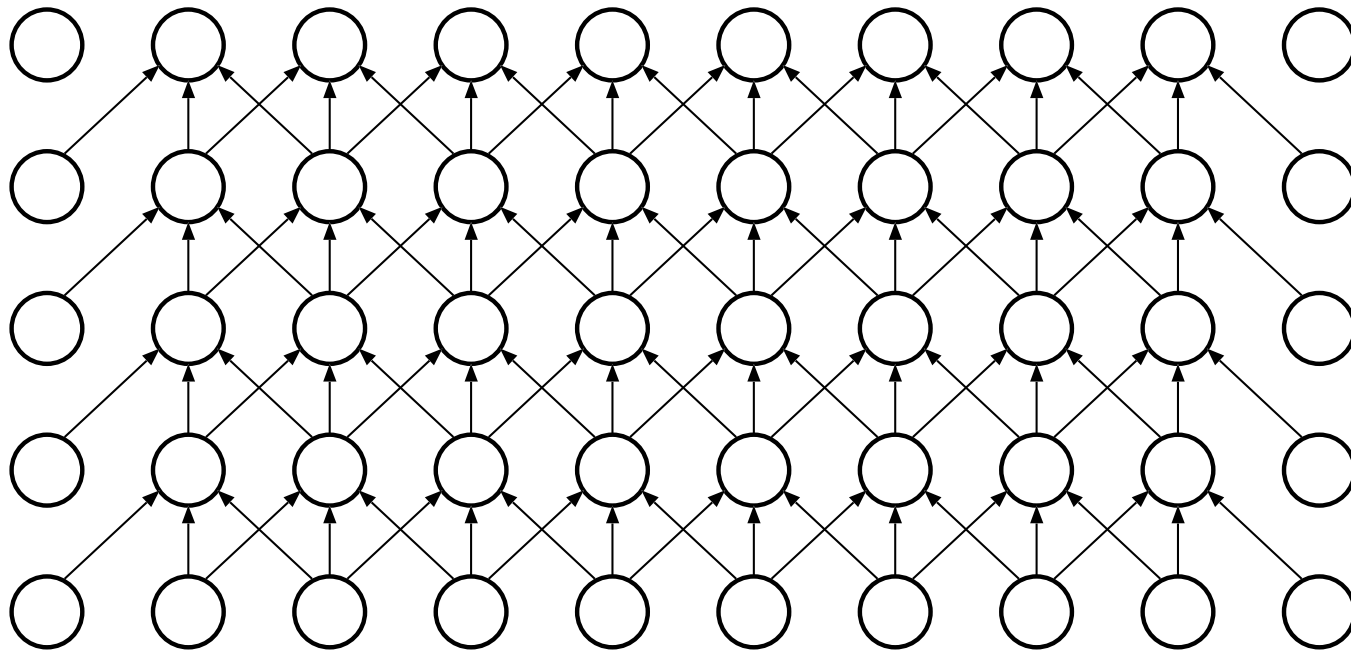
- Communication

$$T(x,t) = r \cdot T(x-1,t-1) + (1 - 2r) \cdot T(x,t-1) + r \cdot T(x+1,t-1)$$



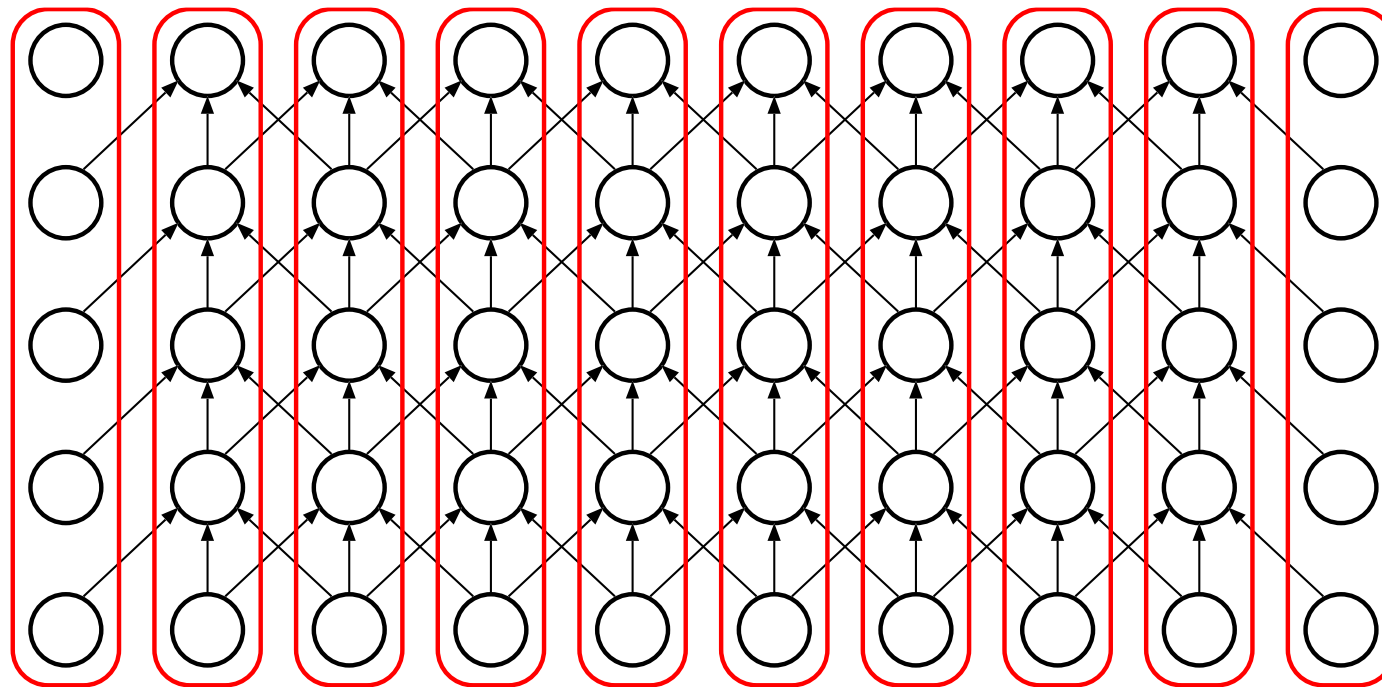
Boundary Value Problem

- Agglomeration



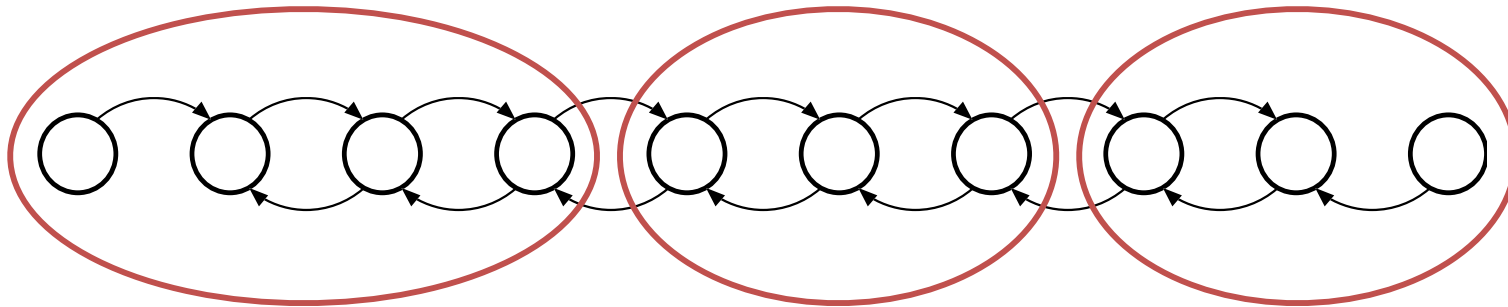
Boundary Value Problem

- Agglomeration

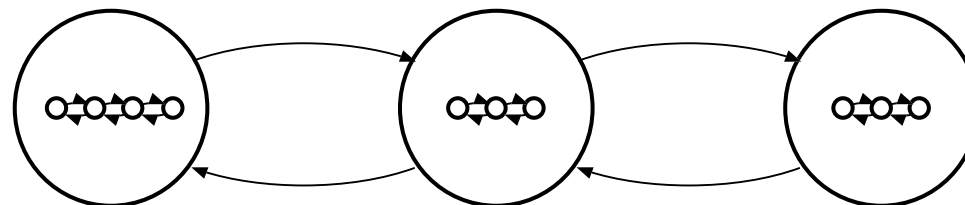


Boundary Value Problem

- Agglomeration



- Mapping



Boundary Value Problem

- Analysis of execution time

- C : time to compute

$$T(x,t) = r \cdot T(x-1,t-1) + (1 - 2r) \cdot T(x,t-1) + r \cdot T(x+1,t-1)$$

- D : time to send/receive one $T(x,t)$ from another processor
- p : number of processors

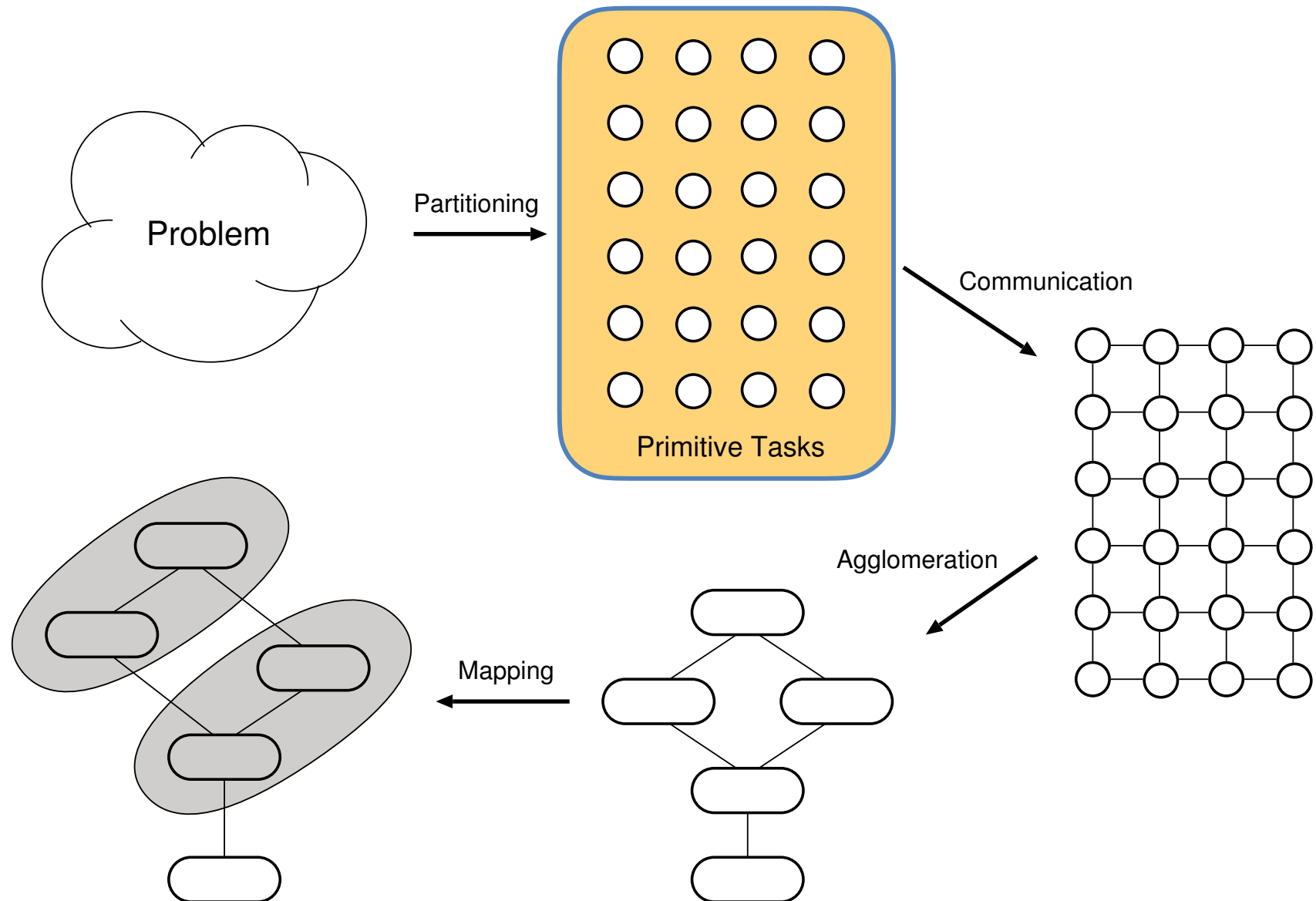
- Sequential algorithm:

- Parallel algorithm:

- Computation per time instant:
- Communication per time instant:

Total:

Foster's Design Methodology



Foster's Design Methodology: Partitioning

- **Partitioning**: Process of dividing the computation and data into many small **primitive tasks**
- Strategies (no single universal recipe...)
 - Data decomposition
 - Functional decomposition
 - Recursive decomposition
- Checklist
 - $>10 \times P$ primitive tasks than P processors
 - Minimize redundant computations and redundant data storage
 - Primitive tasks are roughly the same size
 - Number of tasks grows naturally with the problem size

Partitioning: Data Decomposition

- Appropriate data partitioning is critical to parallel performance
- Steps
 - Identify the data on which computations are performed
 - Partition the data across various tasks
- Decomposition can be based on
 - Input data
 - Output data
 - Input + output data
 - Intermediate data

Partitioning: Input Data Decomposition

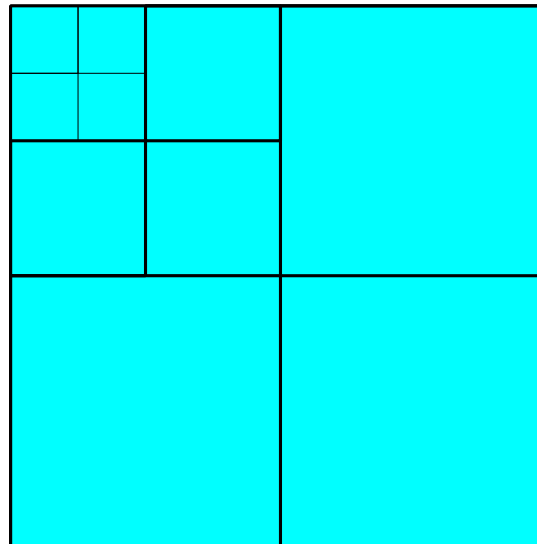
- Applicable if each output is computed as a function of the input
- May be the only natural decomposition if output is unknown
 - Problem of finding the minimum in a set or other reductions
- Associate a task with each input data partition
 - Task performs computation on its part of the data
 - Subsequent processing combines partial results from earlier tasks

Partitioning: Output Data Decomposition

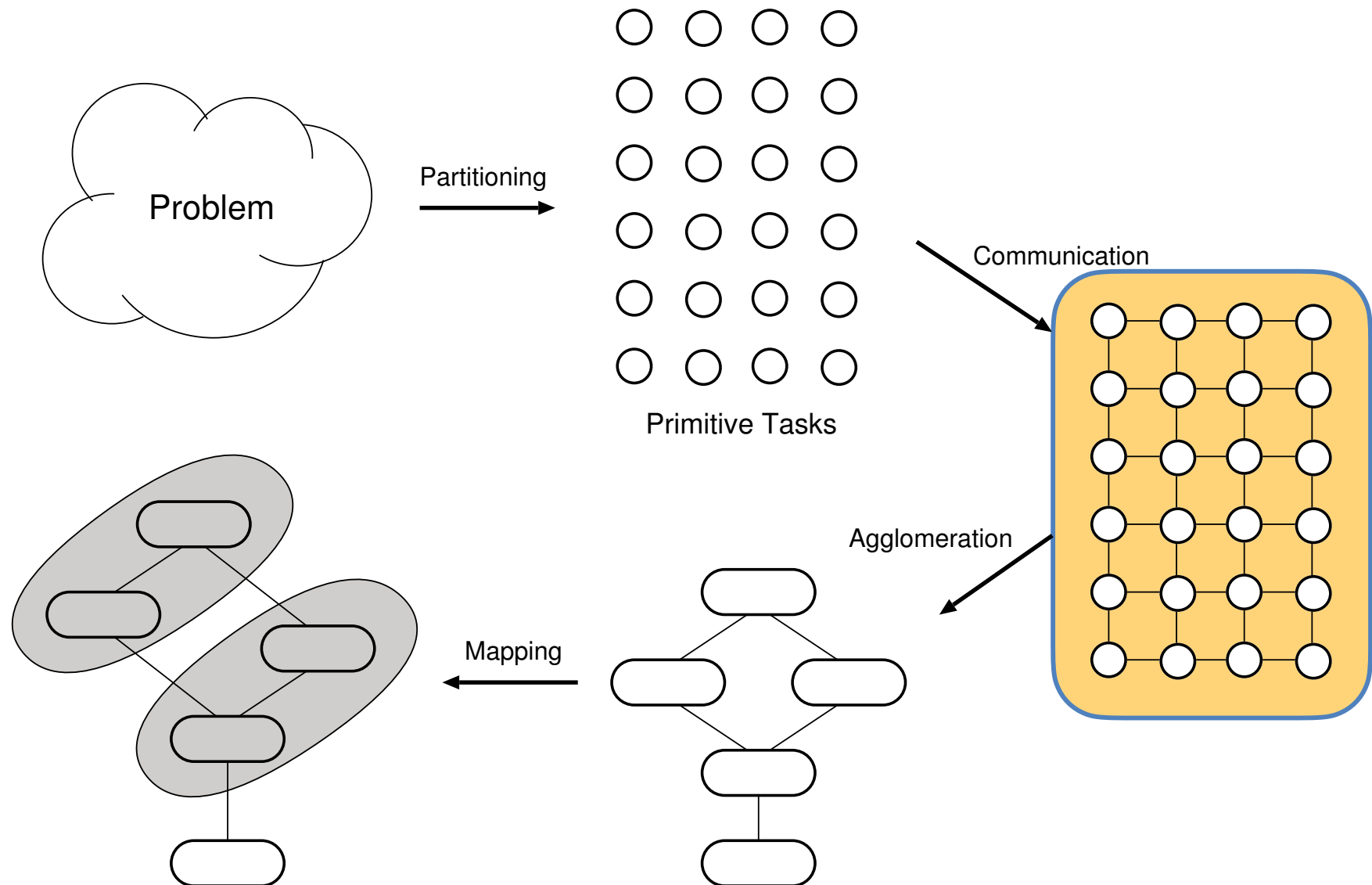
- Applicable if each element of the output can be computed independently
- Partition the output data across tasks
- Have each task perform the computation for its outputs
- Example: Matrix-vector multiplication

Partitioning: Recursive Decomposition

- Suitable for problems solvable using divide-and-conquer
- Steps
 - Decompose a problem into a set of sub-problems
 - Recursively decompose each sub-problem
 - Stop decomposition when minimum desired granularity reached



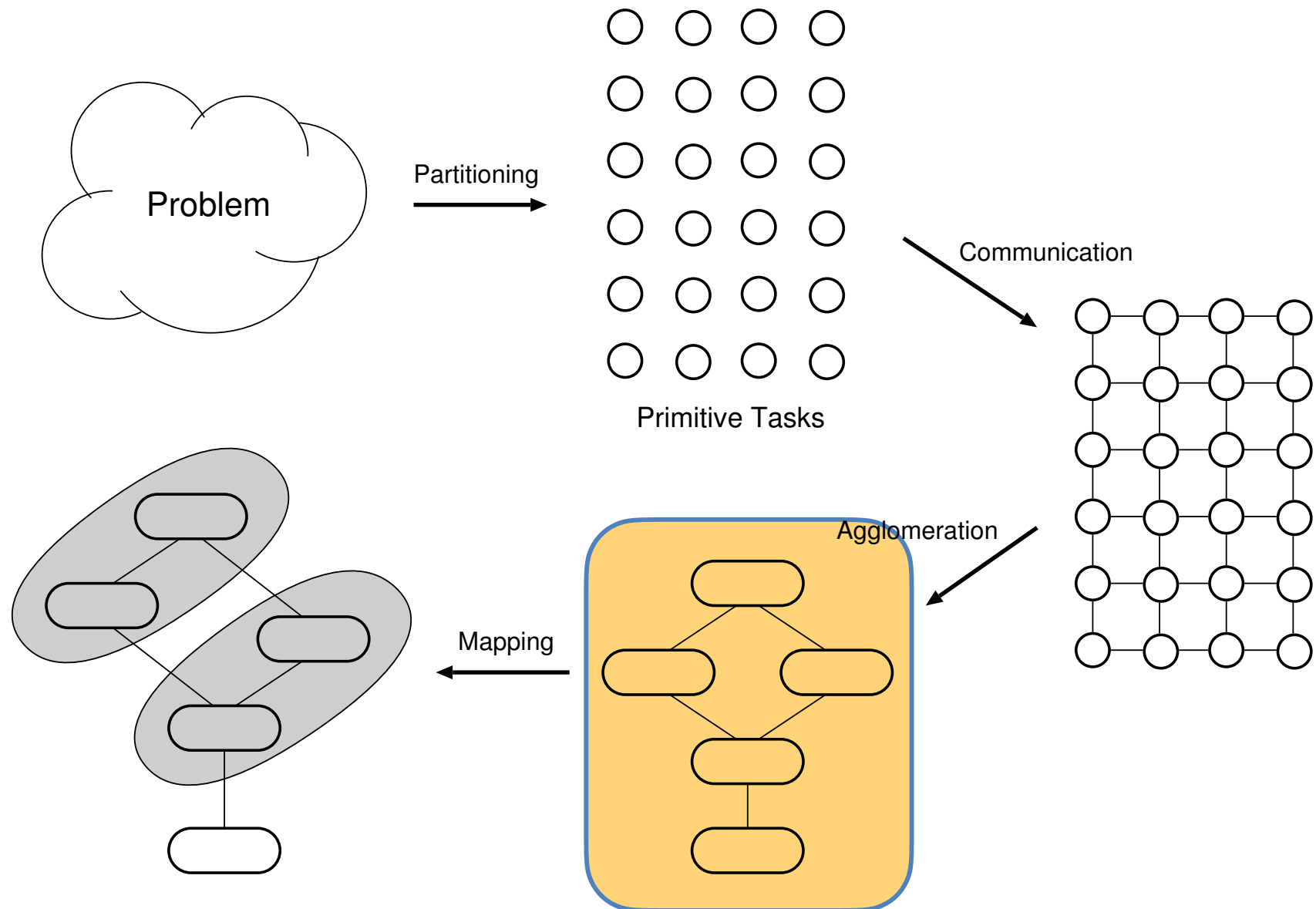
Foster's Design Methodology



Foster's Design Methodology: Communication

- **Communication:** identification of the communication pattern among primitive tasks
 - **Local communication:** values shared by a small number of tasks
 - Draw a channel from producing task to consumer tasks
 - **Global communication:** values are required by a significant number of tasks
 - While important, not useful to represent in the task/channel model
- Checklist
 - Communication balanced among tasks
 - Each task communicates with a small number of tasks
 - Tasks can perform their communication concurrently
 - Tasks can perform their computations concurrently

Foster's Design Methodology



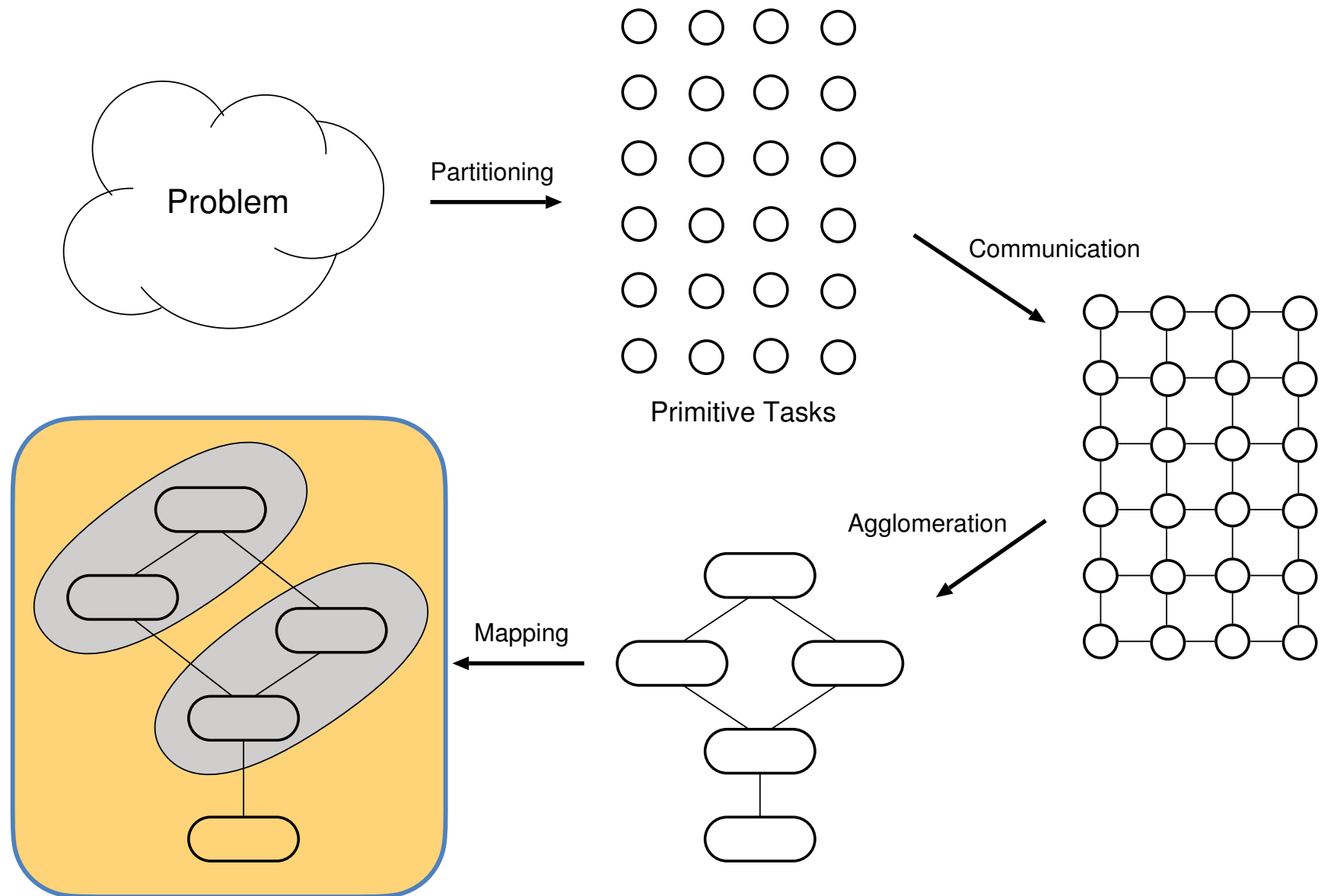
Foster's Design Methodology: Agglomeration

- **Agglomeration:** group primitive tasks into larger tasks
- Strategies
 - Group tasks that have high communication with each other
 - Group sender tasks and group receiving tasks
 - Group tasks to allow re-use of sequential code

Foster's Design Methodology: Agglomeration

- **Agglomeration:** group primitive tasks into larger tasks
- Checklist
 - Locality has been maximized
 - Tasks are balanced in terms of computation and communication
 - Number of tasks grows naturally with problem size
 - Number of tasks is small, but at least as great as P
 - Cost of modifications to sequential code is minimized
 - Consider replication:
 - Replicated computations take less time than the communications they replace
 - Replicated data is small enough to allow algorithm to scale

Foster's Design Methodology



Foster's Design Methodology: Mapping

- **Mapping:** process of assigning tasks to processors
- Strategies
 - Maximize processor utilization (average % time processor are active)
 - ➔ even load distribution
 - Minimize inter-processor communication
 - Map tasks with channels among them to the same processor
 - Take into account network topology

Finding the Maximum

- Determine the maximum over a set of n values
 - reduction operation

$$a_0 \oplus a_1 \oplus a_2 \oplus \cdots \oplus a_{n-1}$$

where $\oplus$ can be any associative binary operation

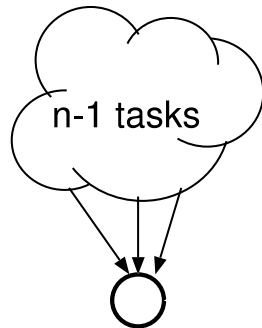
➔ A reduction always takes $\Theta(n)$ time on a sequential computer

Finding the Maximum

- Partitioning
 - Make checking each value a primitive task
 - 1-dimensional domain decomposition
 - One task will compute final solution: root task

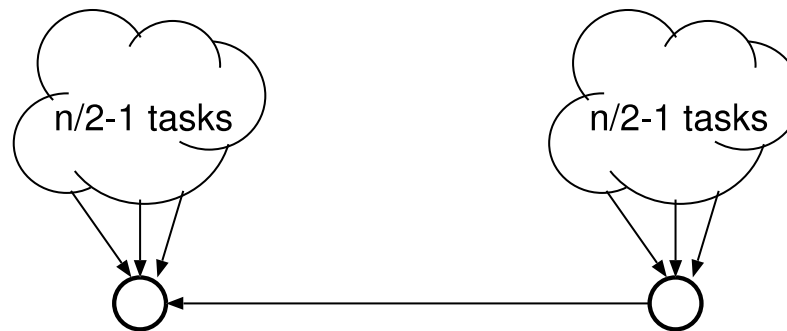
Finding the Maximum

- Communication



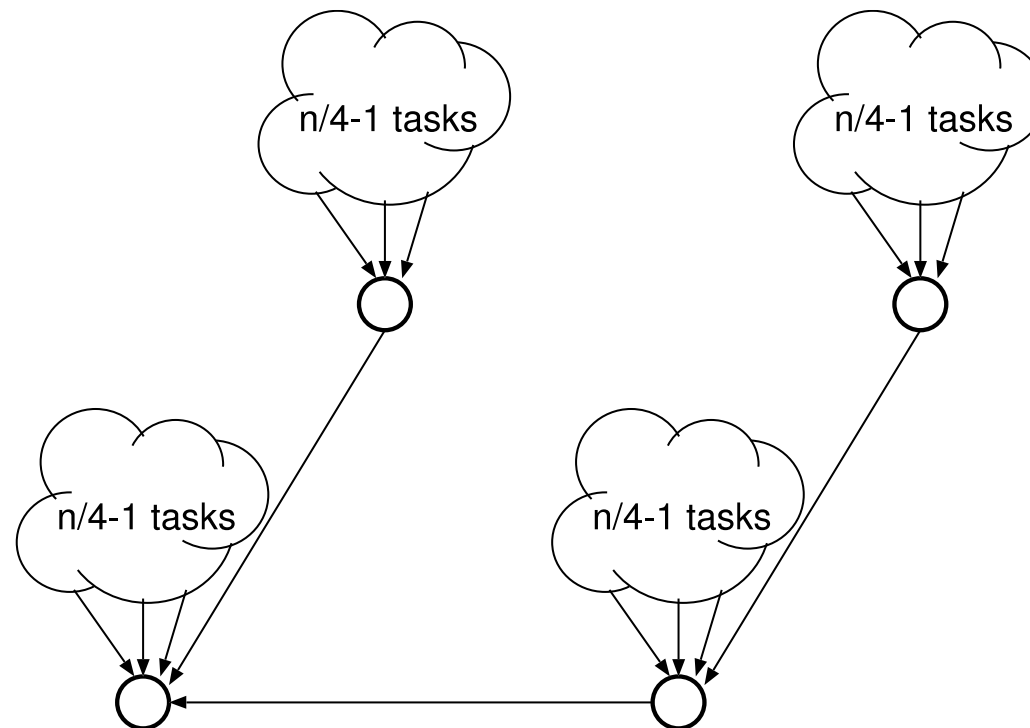
Finding the Maximum

- Communication



Finding the Maximum

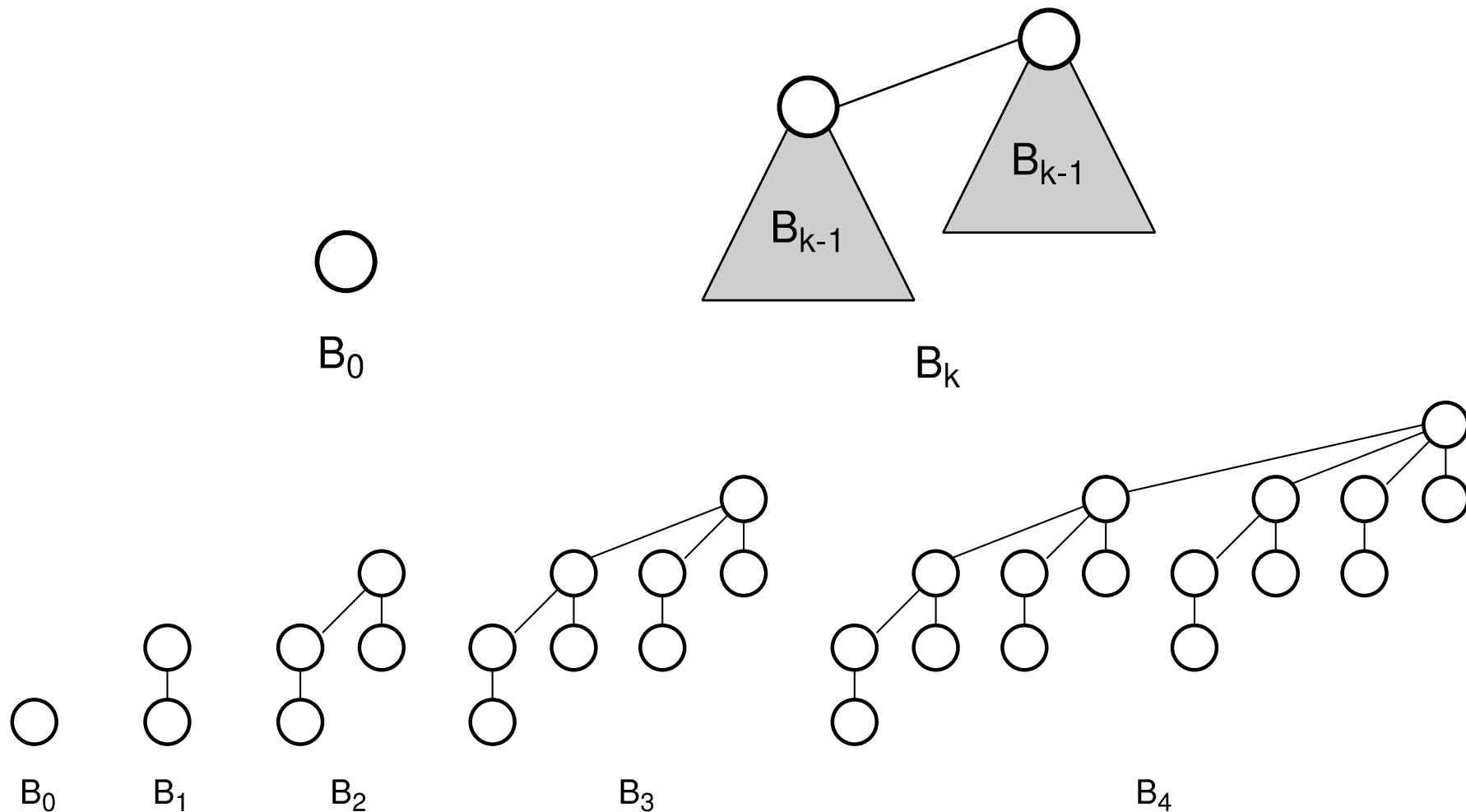
- Communication



– Continue recursively: **Binomial Tree**

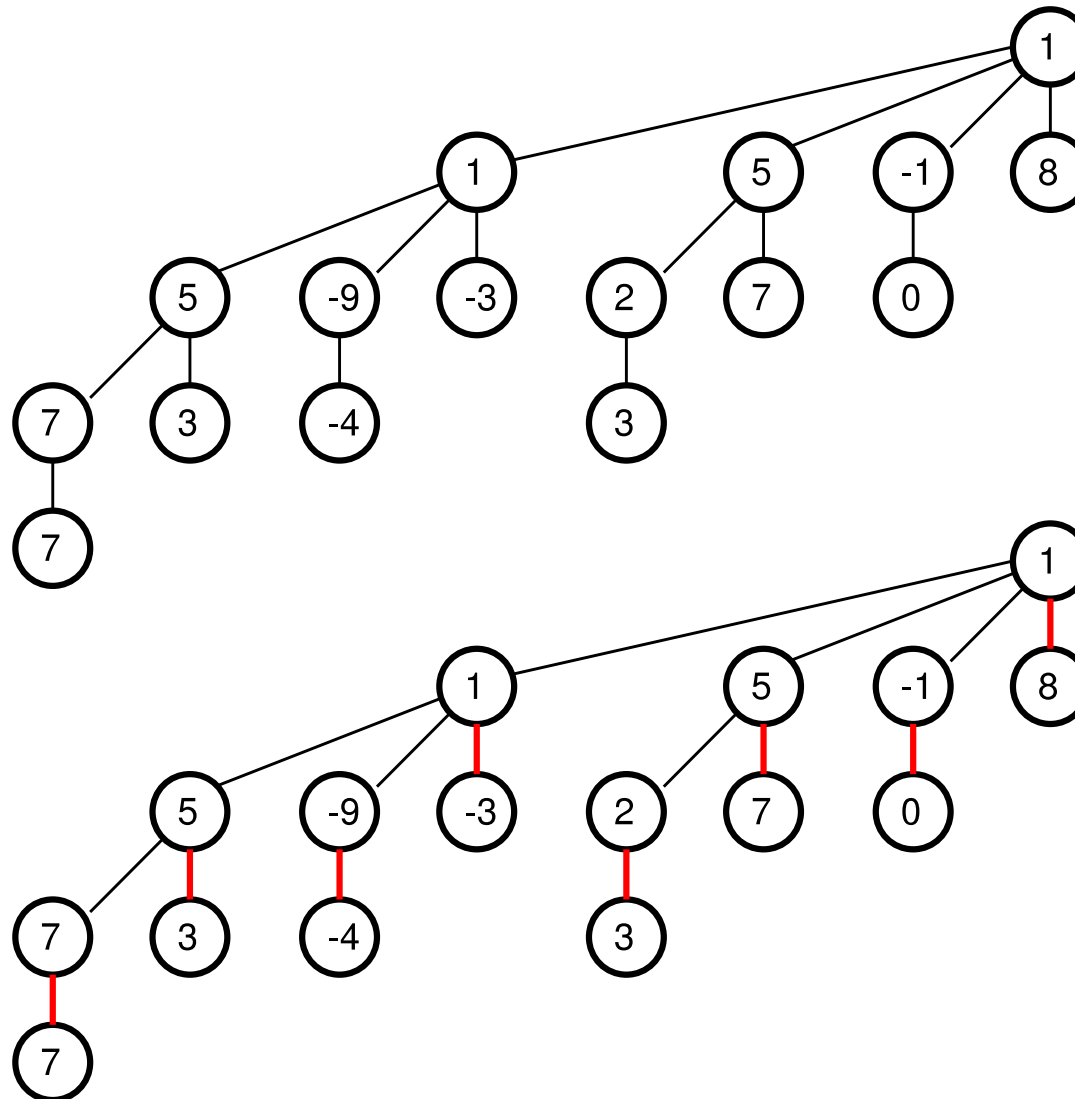
Binomial Trees

- Recursive definition of **Binomial Tree**, with $n=2^k$ nodes



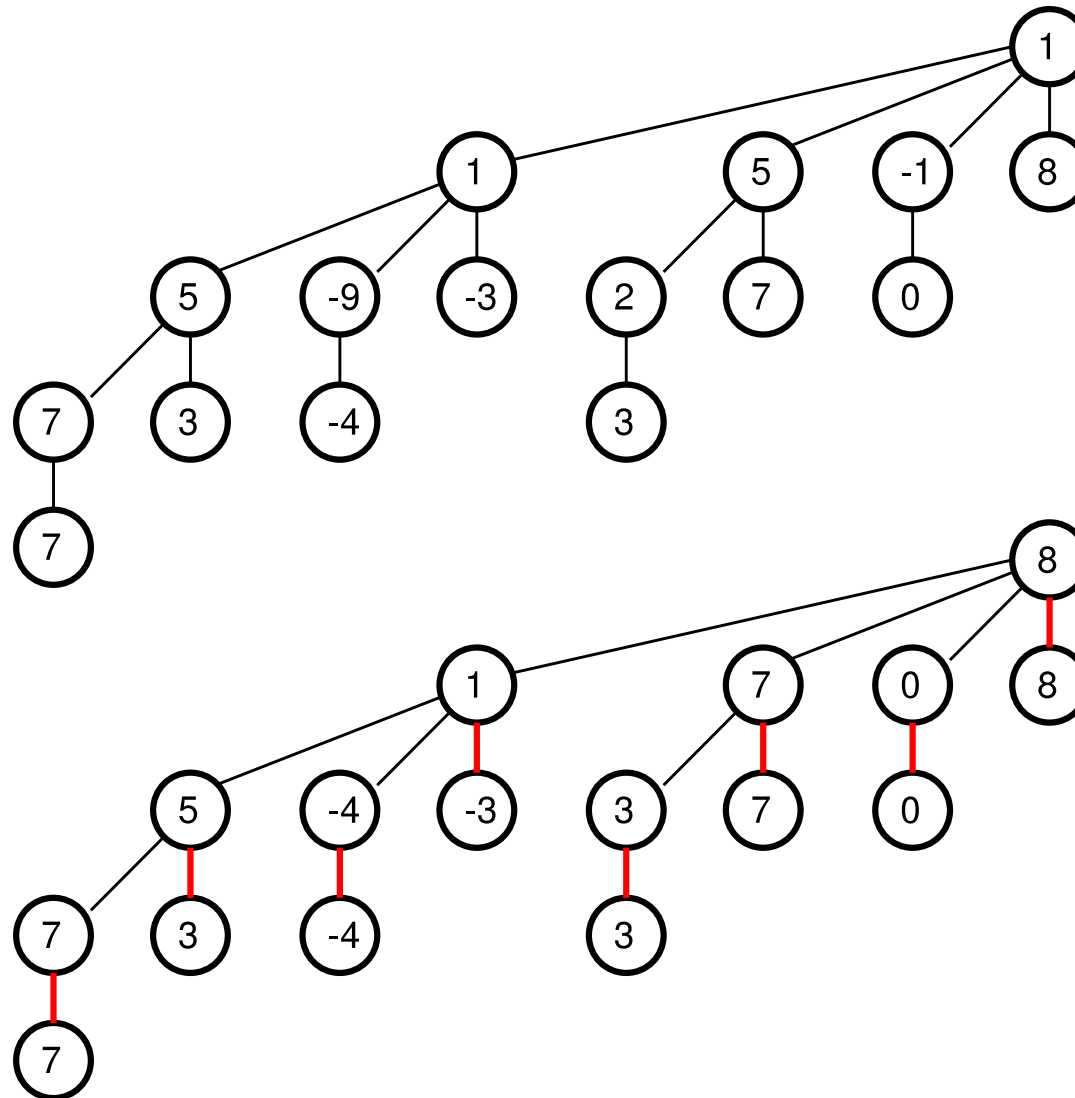
Finding the Maximum

- Illustrative example



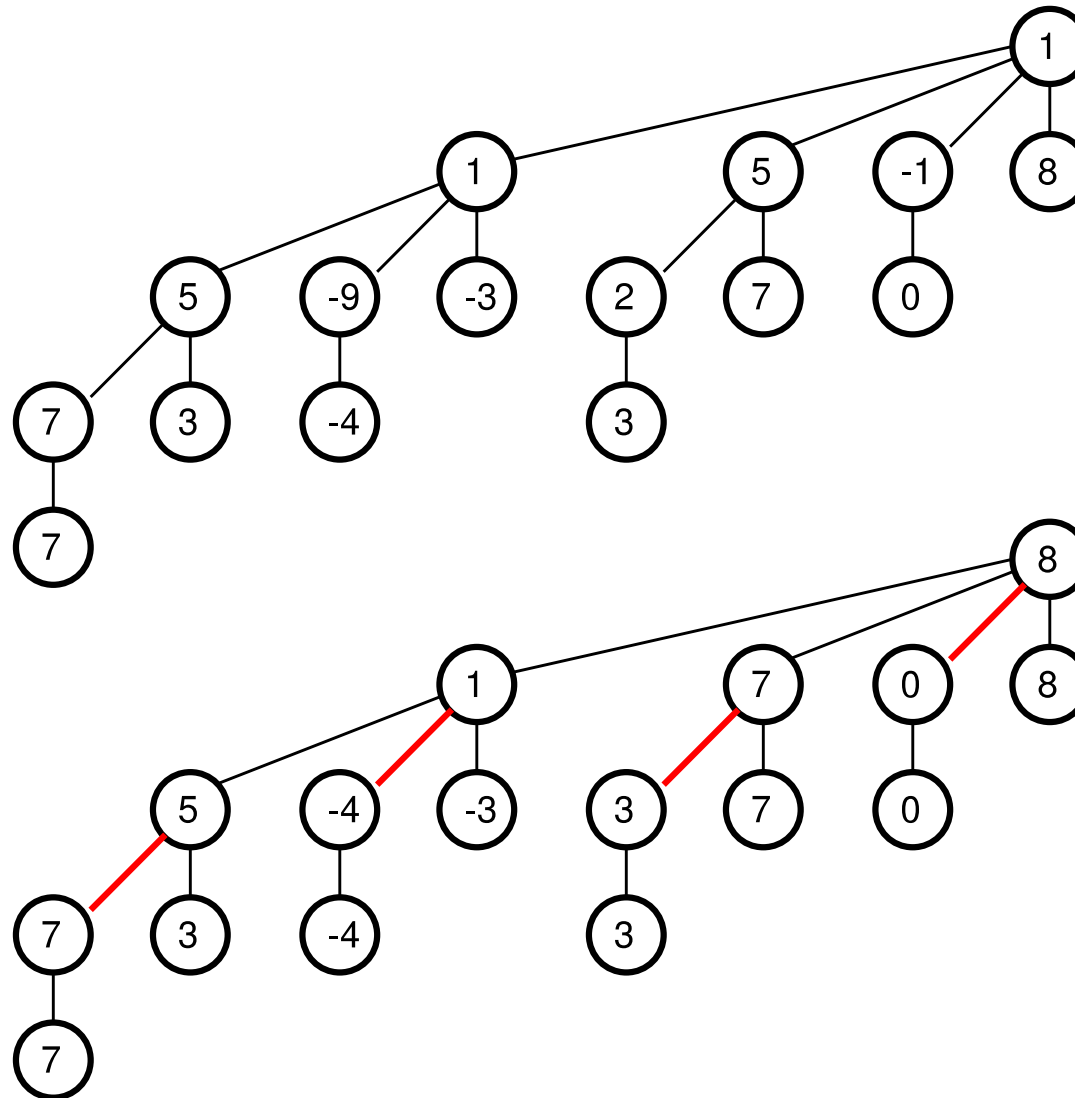
Finding the Maximum

- Illustrative example



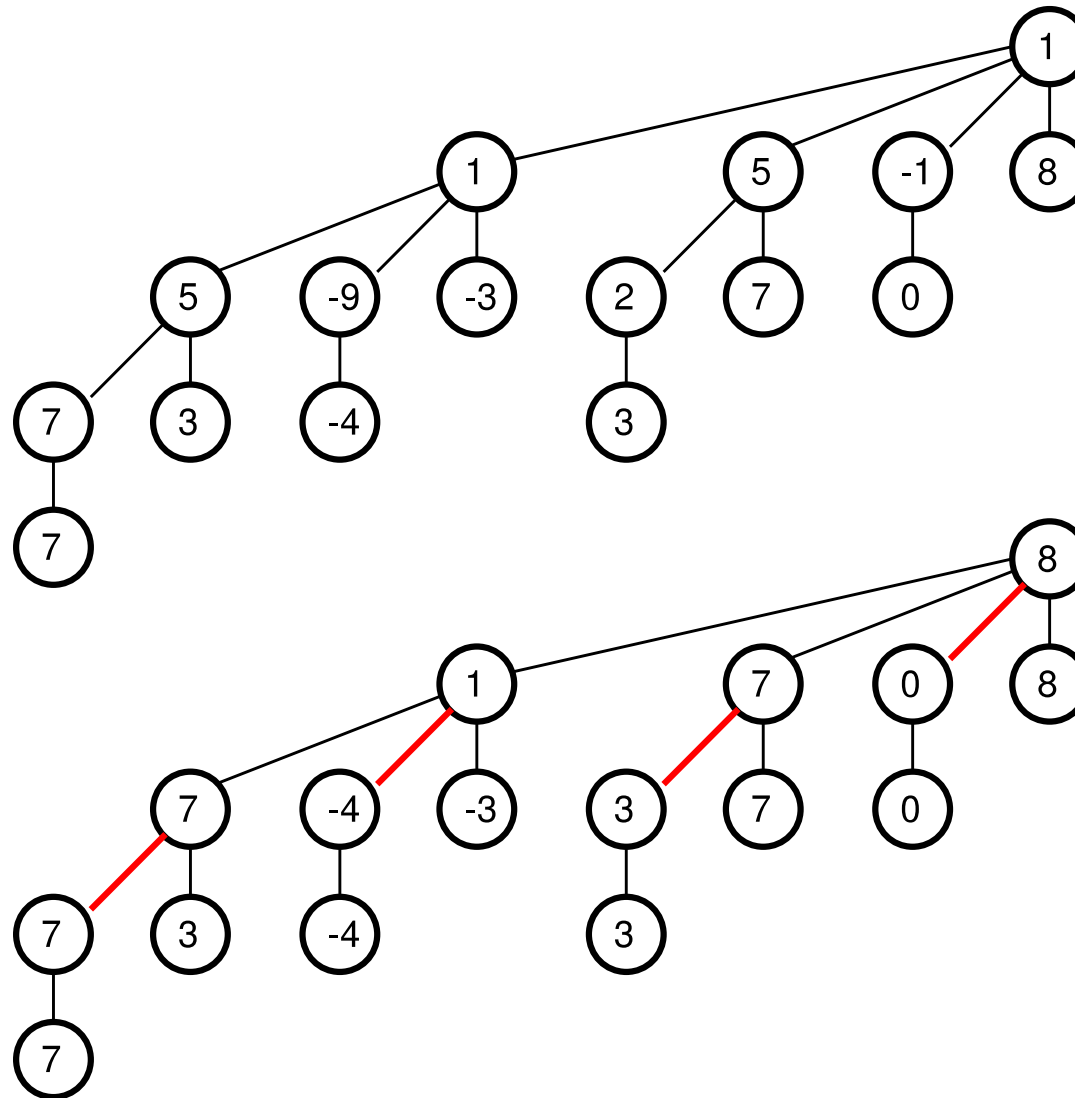
Finding the Maximum

- Illustrative example



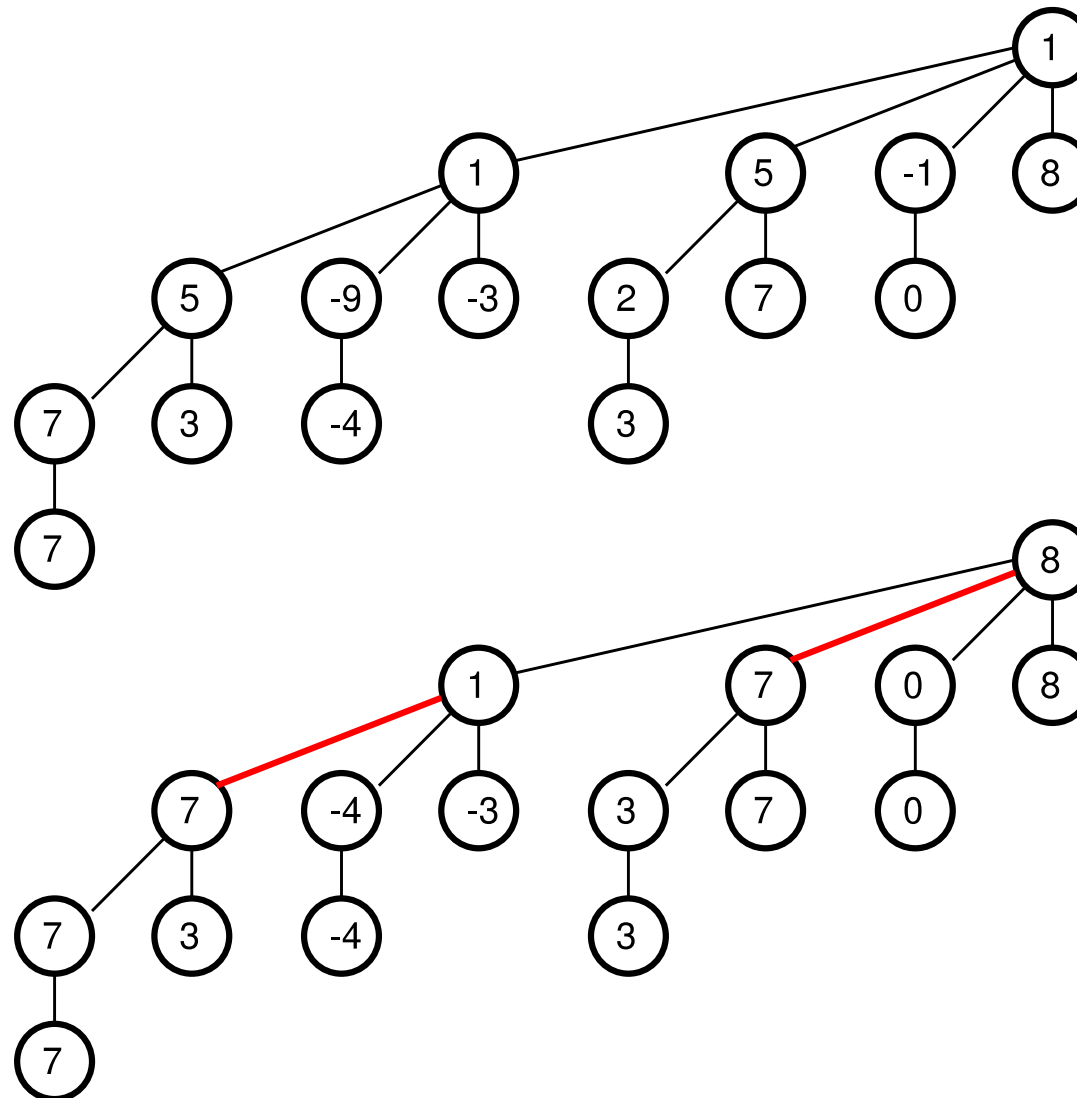
Finding the Maximum

- Illustrative example



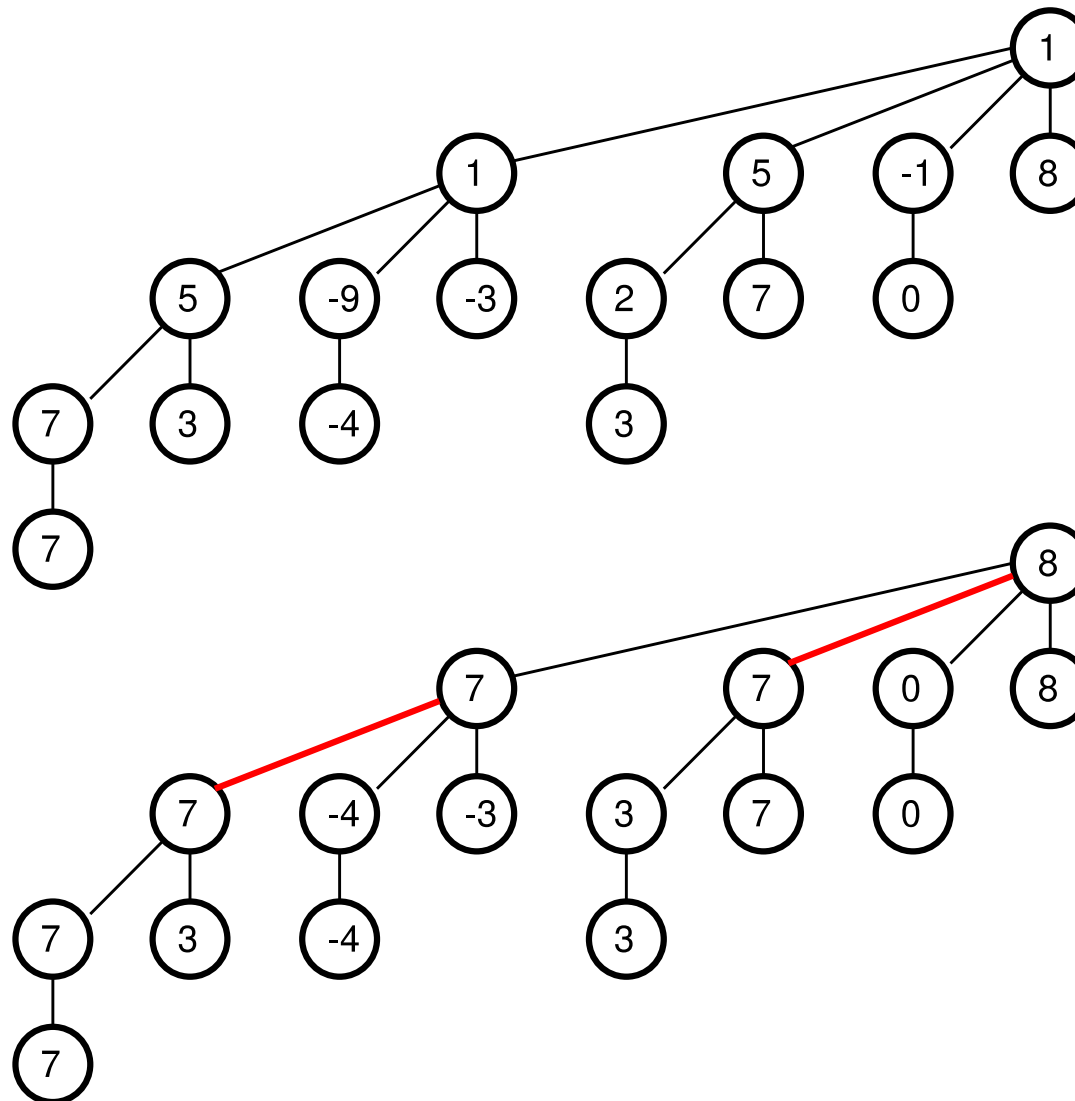
Finding the Maximum

- Illustrative example



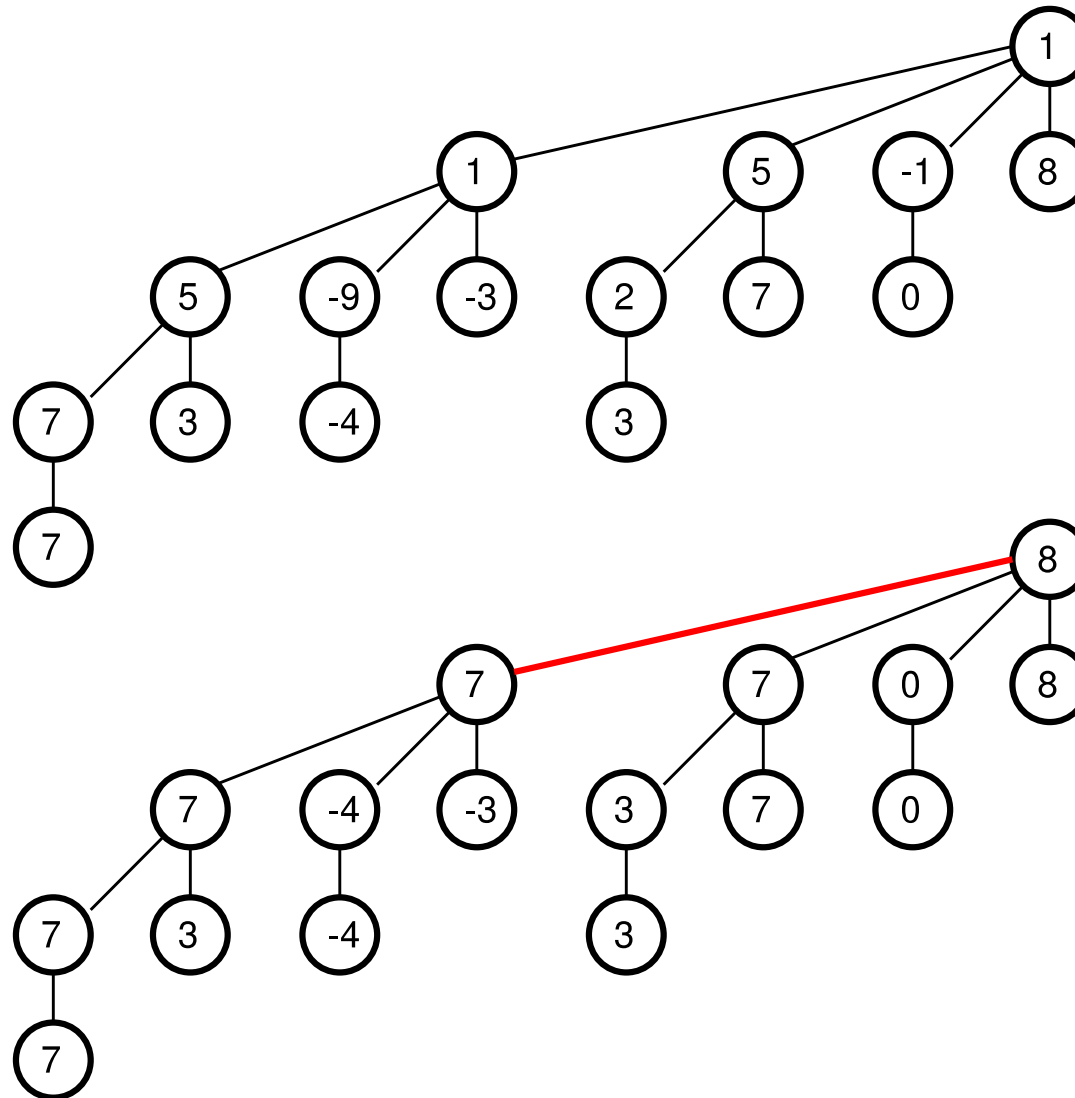
Finding the Maximum

- Illustrative example



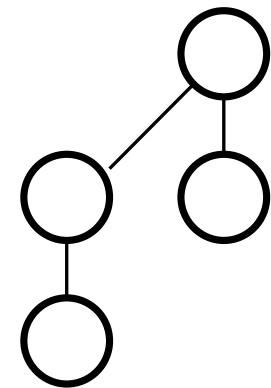
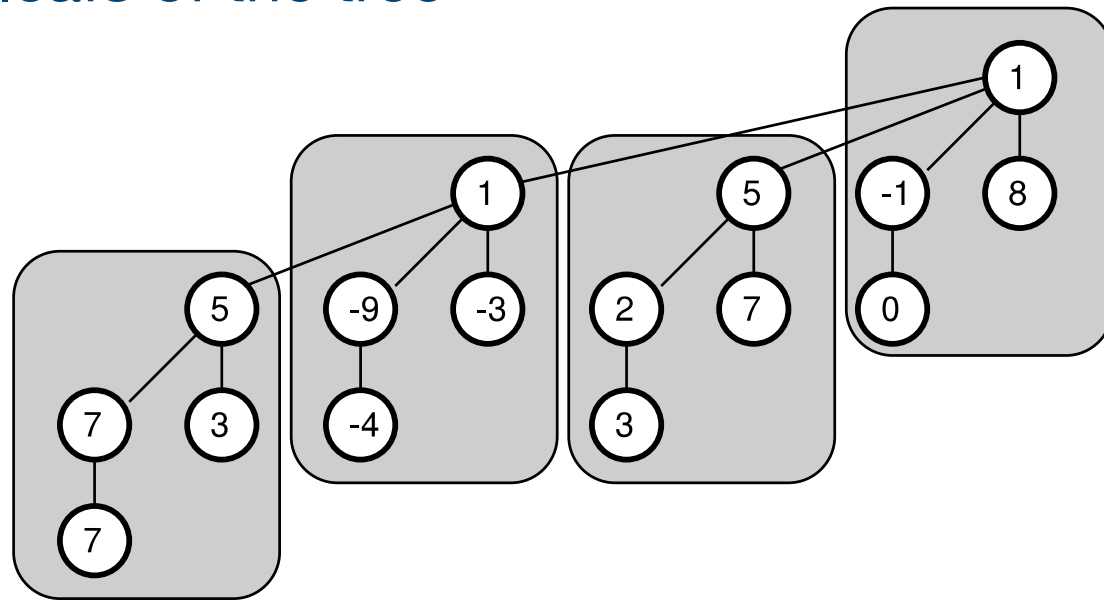
Finding the Maximum

- Illustrative example



Finding the Maximum

- Agglomeration
 - Group n leafs of the tree



- Mapping
 - The same as in the agglomeration phase, use n such that you end up with p tasks

Finding the Maximum

- Analysis of execution time
 - C : time to perform the binary operation (maximum)
 - D : time to send/receive one value from another processor
 - p : number of processors
- Sequential algorithm:
- Parallel algorithm:
 - Computation in the leafs:
 - Computation up the tree:

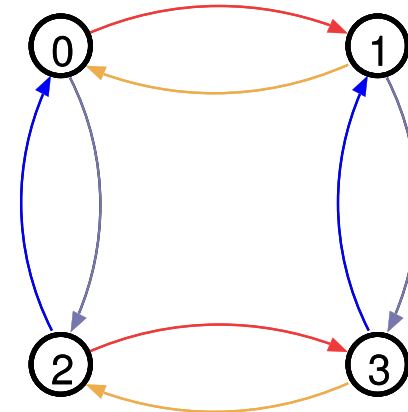
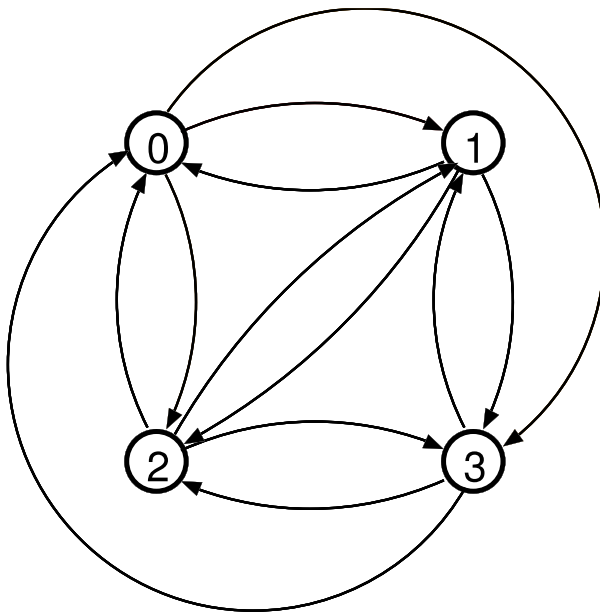
n-Body Problem

- Simulate the motion of n particles of varying masses in two dimensions
 - Compute new positions and velocities
 - Consider gravitational interactions only
 - Straightforward sequential algorithms solve this problem in n^2 time
- Partitioning
 - Make each particle a primitive task
 - 1-dimensional domain decomposition

n-Body Problem

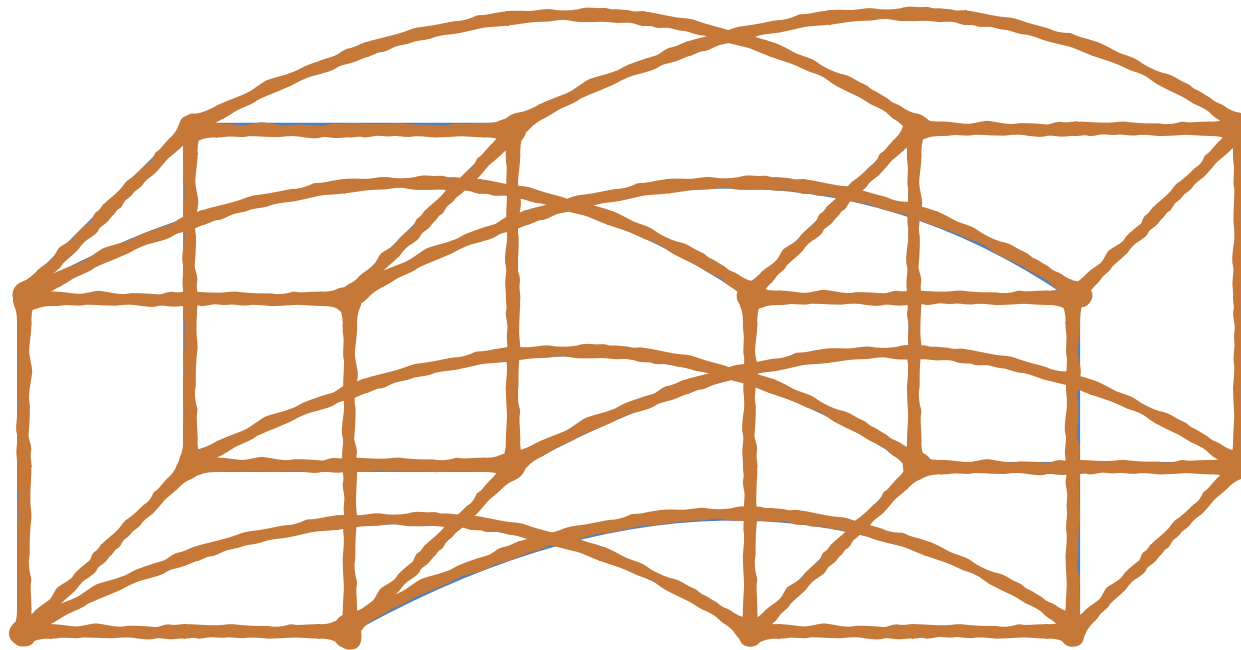
- Communication

- All tasks need to communicate with all tasks!
 - **Gather operation:** one task receives a dataset from all tasks
 - **All-gather operation:** all tasks receive a dataset from all tasks



- Implement communication using a hypercube topology!

Communication on the Hypercube



n-Body Problem

- Agglomeration / Mapping
 - All primitive tasks have the same computation cost and communication pattern
 - No particular strategy for agglomeration required
 - Agglomerate n/p primitive tasks, so that we have one task per processor

n-Body Problem

- Analysis of execution time for one time-step
 - C : time to compute new particle position and velocity
 - D : time to initiate message
 - B : (bandwidth) # of data units that can be sent in one unit of time
 - p : number of processors
 - Sequential algorithm: $Cn(n - 1)$
 - Parallel algorithm:
 - Computation time: $C \frac{n}{p}(n - 1)$
 - Communication time:
 - One message with k data units: $D + \frac{k}{B}$
 - Number of dimensions: $\log p$
 - Length of message at step i : $2^{i-1} \frac{n}{p}$
- Total: $D \log p + \frac{n}{p} \left(\frac{p-1}{B} + C(n - 1) \right) \sum_{i=1}^{\log p} \left(D + \frac{2^{i-1}n}{pB} \right) = D \log p + \frac{n(p-1)}{pB}$

Review

- Programming Distributed-memory Systems
- Foster's Design Methodology
 - Partitioning
 - Communication
 - Agglomeration
 - Mapping
- Application Examples
 - Boundary value problem
 - Finding the maximum
 - n-body problem

Next Class ...

- MPI
- Application example